

Pythia：利用工作流的可预测性实现高效的智能体原生 LLM 服务

Shan Yu^{1*}, Junyi Shu^{1*†}, Yuanjiang Ni², Kun Qian², Xue Li³, Yang Wang⁴, Jinyuan Zhang¹, Ziyi Xu⁵, Shuo Yang⁶, Lingjun Zhu³, Ennan Zhai², Qingda Lu², Jiarong Xing⁷, Youyou Lu⁸, Xin Jin⁹, Xuanzhe Liu⁹, Harry Xu¹

¹UCLA ²Alibaba Cloud Computing ³Alibaba Group ⁴Intel ⁵SJTU
⁶UC Berkeley ⁷Rice University ⁸Tsinghua University ⁹Peking University

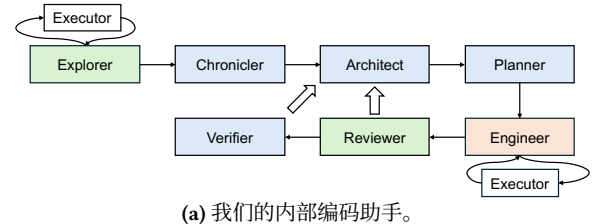
摘要

随着 LLM 应用日益复杂，开发者越来越多地采用多智能体架构，将工作流分解为专门化且相互协作的组件。这引入的结构约束了智能体行为，也暴露出有用的语义可预测性。传统 LLM 服务运行在高度动态、不确定的条件下，而这种结构化拓扑为减少运行时不确定性提供了机会；然而，现有系统未能利用它，仍将智能体工作负载视为一般流量，造成显著低效。我们分析智能体服务平台及内部编码助手的生产轨迹，发现了前缀缓存命中率低、长上下文请求引发严重资源争用，以及扩缩容策略欠佳导致大量排队等关键瓶颈。为此，我们提出多智能体服务系统 Pythia，它通过服务层的简单接口捕获工作流语义，打开新的优化空间，相较最先进的基线显著改善吞吐量和作业完成时间。

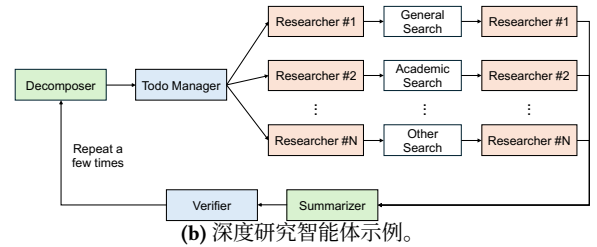
1 引言

一般 LLM 服务领域的传统认识是，一切都不可预测：既无法预知任一时刻会激活哪些类型、多少个模型，也无法预知每个模型接收的请求类型和数量。这种不确定性源于模型提供方通常只向用户开放访问模型的 API 端点，无法控制模型何时以及如何被调用。因此，LLM 服务的大多数既有工作 [1–4] 都假设执行环境高度动态，服务系统几乎所有配置参数（e.g. KV 缓存大小、共享模式或自动扩缩容水平）都必须根据运行时观测的负载自适应调整。这种适应性提供了很大的灵活性，但完全被动响应的服务系统往往收敛到次优策略。

生产轨迹分析。为说明现有黑盒方法如何限制效率，我们分析了智能体服务的大规模生产轨迹，并深入剖析一个内部多智能体编码助手。分析 (§2) 揭示了智能体负载服务中的三个根本挑战，它们与常见假设相悖。第一，缓存效果远低于预期：专门化智能体使用不同的提示，调用之间存在较长时间间隔，使先前缓存的前缀被逐出，频繁出现零命中和很长的预填充时延。第二，调度不考虑工作流，导致资源效率严重不足：异构请求被任意混合，在副本和模型之间造成负载失衡，并引发内存争用、频繁抢占和高成本重计算。第三，多智能体工作流呈现急剧且有结构的突发，一分钟内的增幅常超过 50%，使服务容量不堪重负；现有系统依赖被动、平滑化的自动



(a) 我们的内部编码助手。



(b) 深度研究智能体示例。

图 1: 多智能体工作流示例。

扩缩容，无法感知工作流依赖，也就无法预见这些级联突发，最终导致多个 LLM 排队和拥塞。

关键洞见：多智能体工作流可预测。与本质上不可预测的一般 LLM 服务相比，多智能体工作流（见图 1）呈现截然不同的情况。底层模型可能是通用模型，但固定的智能体类型集合及其在工作流中的具体角色，严格约束了这些模型的调用方式。

在这类工作流中，高层推理组件通常由能力较强的 LLM 充当架构师、规划者或任务分解者，处理复杂任务要求，并将其拆成一组子任务。随后，每个子任务交给配备特定模型（通常较小）和一组工具的专门工作智能体（e.g. 工程师或研究者）。每个智能体通常执行明确的任务，并以一致方式与模型交互。在许多工业部署中，GPU 集群专门服务少量特定的智能体应用（e.g. 编码助手、深度研究系统）。

因此，来自同一智能体的请求倾向于遵循相似的使用模式，从而引入相当程度的资源使用可预测性。

多智能体应用中的资源可预测性来自两个关键因素：(1) 工作流可预测及 (2) 负载可预测。首先，在服务时，工作流结构通常用图表示，节点对应各智能体，边表示数据流；该结构要么已完全确定（静态工作流），要么遵循重复模式（通过推理动态演变的工作流）。这种结构信息揭示请求如何在智能体之间传播，使优化能

*Shan Yu 与 Junyi Shu 贡献相同。

†Junyi Shu 为通信作者。

够依据更充分的信息。例如，每次都处理全新输入的工作智能体，几乎无法从前缀缓存中获益。同样，当智能体部署在多个 GPU 上时，调度可用于迅速填满流水线并最大化利用率，无需依赖仅优先处理先到请求的朴素 FCFS 策略。

其次，每个工作智能体（非编排者）负责特定任务，复杂度存在界限；复杂问题通常被分解为多个较简单的子任务，因此每个智能体节点上的请求输出长度和执行时间，通常遵循相对简单的分布，可通过预运行或在线性能剖析准确估计。在表征这些分布后， workflow 执行期间便可利用估计值，在调用智能体前准确地预热缓存并主动进行模型自动扩缩容。

Pythia。基于这一洞见，我们设计了智能体原生服务系统 Pythia，系统地利用 workflow 可预测性，实现此前被视为难以处理的优化。传统服务引擎把智能体推理视为完全不透明且动态的过程，因此只能被动地进行次优资源分配。相较之下，Pythia 根据请求在 workflow 图中的当前位置，以及其特定智能体角色预期产生的输出量，近似估计请求的剩余执行时间，以实现主动的全局协调。系统将结构信息转换为按预期完成时间调度请求的优先级机制，由此实现忽略 workflow 的系统所无法完成的细粒度优化，显著改善端到端性能和集群利用率。

挑战。要充分利用这些可预测性来高效服务多智能体应用，必须克服以下两个挑战。

第一个挑战是如何告知服务系统 workflow 的哪些方面可预测。例如，workflow 图是必须传递给服务系统的关键信息。然而，智能体开发正变得越来越普及（e.g. OpenClaw 所展示的情况 [5]），许多智能体开发者可能并非编程专家，因而不完全理解或无法指定底层图结构。实际多智能体应用还常生成动态图，编排者根据初始任务的复杂度创建智能体节点，这使问题更加困难。

为解决这一问题，Pythia 提出核心贡献：基于示例的合成方法 [6]，从性能剖析数据自动合成图表示。这一方法建立在两个重要观察之上。首先，多数智能体应用结构简单，使用代表性输入执行几次，就足以生成帮助构造 workflow 图的性能剖析数据。其次，动态图虽然无法静态描述，却通常由重复模式组成（e.g. 图 1a 中连续的 Engineer-Executor 交互）。分析这些模式后，便可使用正则表达式完整表达并预测动态执行流。

因此，Pythia 只要求智能体框架 [5, 7–9] 通过兼容 OpenAI 的端点已经支持的 `extra_body` 载荷 [10]，传入应用和智能体类型的轻量信息，无需修改用户应用代码。在网关，Pythia 利用性能剖析器提取 workflow 和负载信息，包括捕获 workflow 图的正则表达式以及各智能体的输出长度，并将这些注解重新写入载荷，供下游服务系统使用。

第二个挑战是如何在服务系统中利用这些预测信息。传统 LLM 服务引擎根据当前系统状态管理内存和调度请求，采用被动响应范式。直接转向主动预测模式可能带来严重性能损失：如果管理不当，激进的前瞻决策容易触发破坏性的资源抖动和过早缓存逐出。为降低风险并稳妥实现优化，Pythia 引入三个协调机制，分别对应轨迹分析识别的关键机会 (§2)，以主动优化执行。在节点层面，推测式缓存管理器用受 Belady 启发的策略增强

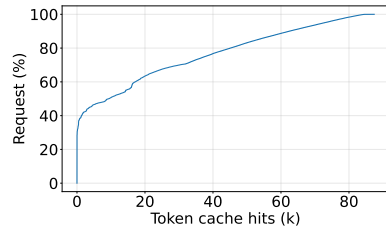


图 2: 智能体服务平台中，不同 Token 命中数量所对应请求比例的累积分布函数（CDF）。

被动 LRU，利用 workflow 信息预取共享上下文、逐出短暂使用的 Token，并异步预热缓存以掩盖预填充时延。在集群层面，前瞻请求调度器利用预测输出长度和 workflow 结构平衡异构负载，并确定执行优先级，以减少内存争用和队头（HoL）阻塞。与之配合的阶段自适应自动扩缩容器预见扇出及阶段转换等结构性负载变化，主动扩缩模型容量，缓解突发导致的排队并提高整体资源效率。

结果。我们在 SGLang 之上实现 Pythia，并在代表性多智能体应用中评估。结果表明，平均作业完成时间（JCT）的缩短倍数最高达到 2.9×，吞吐量提升 1.96×。Pythia 目前正在内部开展更大集群规模的广泛测试和试点部署，以推动更广泛的使用。

2 背景与动机

2.1 背景

LLM 服务。当今 LLM 服务系统围绕请求级和 Token 级执行构建 [1–3, 11, 12]。性能优化主要针对孤立的微基准，通过缩短首 Token 时间（TTFT）和 Token 间隔时间（TBT）来加快预填充和解码阶段。关键在于，通常假定服务层事先不知道正在处理请求的输出，也不知道即将到来的负载。把每个请求视为不透明、孤立的实体，迫使服务层采用保守、被动的策略。例如，请求调度通常局限于 FCFS 队列，前缀缓存管理则严格依赖被动的、按最近使用情况逐出的策略。

多智能体 workflow。LLM 交互范式正在迅速从孤立且有人参与的聊天会话，转向自主、目标驱动的智能体 [5, 13, 14]。受此转变推动，在过去六个月里，由执行多步 workflow 的智能体产生的推理请求占比急剧增长。在我们的智能体服务平台，当前超过 80% 的请求来自智能体 AI 应用。在现代多智能体系统中，高层目标被分解为可执行计划，具体子任务被分配给专门的工作 LLM 和工具。这些工作智能体由定制系统提示驱动，通常还使用领域专用模型，承担规划、编码和审查等不同角色。传统服务系统将智能体请求当成普通 LLM 请求，处理时不考虑智能体语义。

分层前缀缓存。前缀缓存保存已计算的键值（KV）张量，减少重复预填充。现代系统采用三级层次结构平衡时延与容量 [15, 16]：L1 位于稀缺的 GPU HBM，供执行时立即访问；L2 位于 CPU 主机 DRAM，作为容量较大的中转缓冲；L3 使用共享存储持久保存可跨引擎复用的上下文，并减少全局缓存未命中。

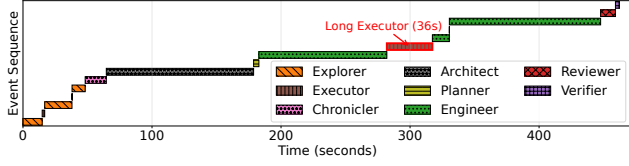


图 3: 编码智能体工作流的时间线: 每个条形表示一个智能体, “executor” 表示命令行执行。

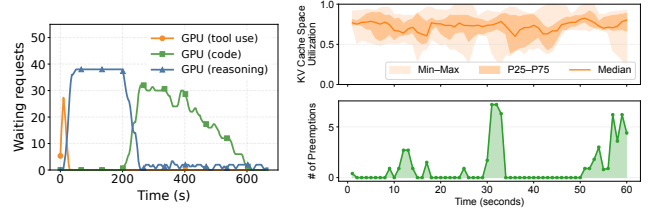
2.2 为多智能体工作流提供 LLM 服务的挑战

随着智能体 AI 日益成为生产负载的主流, 包括我们在内的 MaaS 提供方推出了采用专门计费模式的产品, 以支持这些应用 [17–20]。然而, 架构上的根本不匹配仍然存在: 这些服务仍依赖旧式、不感知应用的推理 API。复杂工作流被展平为一系列不透明、相互独立的请求, 服务层完全不知道结构依赖及不同智能体角色。为说明这种黑盒接口如何限制效率, 我们分析智能体服务的大规模生产轨迹, 并深入剖析部署于其上的内部多智能体编码助手。分析揭示三个主要挑战: 工作流不可见会直接损害系统性能。

挑战 #1: 缓存命中率意外偏低, 造成很长的预填充时间。现有工作常假设智能体 AI 应用的提示结构重复, 因此天然具有很高的缓存命中率 [21–23], 但我们的生产轨迹揭示了明显相反的情况。图 2 给出 Token 缓存命中数量不同的请求分布; 超过 40% 的请求没有命中或命中率很低。原因是专门化智能体依赖不同的系统提示, 无法跨智能体共享前缀。此外, 工作流的顺序执行本身会削弱缓存保留。图 3 展示编码助手的执行时间线。我们观察到: (1) 只有少量连续的 LLM 请求能够从前缀缓存中获益; (2) 这些请求之间可能存在耗时数十秒的长工具调用 (e.g. 代码编译)。到同一智能体再次被调用时, 其此前缓存的前缀大多已被中间流量冲掉。

挑战 #2: 负载失衡及不协调的资源竞争浪费资源。图 4 展示一个 8 GPU 集群上编码智能体的一分钟服务轨迹, 系统受到内存压力及频繁请求抢占的严重限制。低效源于在不了解底层工作流的情况下, 混合从轻量 Planner 响应到包含数千 Token 的 Architect 输出等异构请求。我们识别出三种主要失衡来源: 首先, 朴素路由使同一模型的数据并行副本间存在显著负载偏斜; 其次, 不同模型的资源分配不均, 一些已饱和, 另一些仍利用不足; 最后, 缺乏工作流信息, 使共置智能体随意竞争共享 GPU 内存, 无法协调。结果是过载副本不得不抢占请求; 在不知道预期输出长度时, 系统往往不加区分地逐出较新的请求。被抢占的请求必须从头重计算, 造成大量计算浪费及明显的队头阻塞。

挑战 #3: 突发导致多个 LLM 排队与拥塞。多智能体工作流天然存在很强的结构性波动。在智能体服务平台, 我们观察到定时自动化任务 (e.g. 批量 cron 作业) 可使某些模型的请求量在一分钟内激增 50.3%。如图 5 所示, 这些大幅突发很快使支撑系统的多个 LLM 不堪重负。多智能体工作流的相互依赖还从根本上加剧了问题: 入口节点 (e.g. Explorer 智能体) 的突发不会孤立存在, 而会触发一波级联请求, 迅速传播到下游智能体 (e.g. Chronicler、



(a) 工作流中各节点之间的流量失衡。(b) 服务同一模型的各节点之间的内存失衡。

图 4: 服务多智能体编码助手时的负载失衡和抢占。

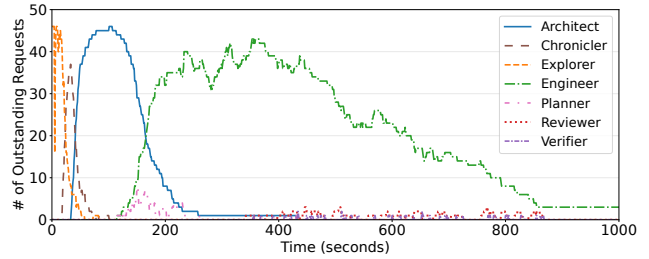


图 5: 一批 cron 作业下, 多智能体编码助手尚未完成的请求数量。

Architect、Engineer)。然而, 现有服务系统依赖经过平滑的长期历史指标, 扩缩容不感知工作流, 无法预见模型间传播。标准的被动自动扩缩容需要数十秒才能增加容量; 在此期间, 突发往往已经移到流水线中的下一个模型, 使系统始终落后于波前, 导致队列深度急剧上升。

2.3 智能体原生服务中的预知能力

为解决上述挑战, 首先需要理解当今推理服务为何无法缓解这些问题。现有 LLM 服务引擎围绕一个隐含假设设计: 请求由彼此独立的人类用户产生。人类聊天流量的意图互不相关, 到达时间不可预测, 提示内容任意。服务系统只能采用严格在线算法, 在不了解未来的情况下, 被动管理缓存、调度请求和扩缩容量。

然而, 智能体流量由程序产生。多智能体工作流遵循特定控制流, 复用既定提示模板, 访问模式由底层源代码决定。虽然确切执行路径可能动态生成, 但可能执行的空间远比人类交互受限。这种程序性质带来根本转变: 如果服务系统能够事先捕获工作流信息, 就能从被动的在线猜测转向主动、接近离线的优化。具体而言, 我们识别出三类关键信息, 可以系统地缓解观测到的瓶颈。

机会 #1: 工作流图。捕获多智能体应用的结构依赖图, 可以实现多个关键系统优化。首先, 它识别反复出现的智能体角色, 使服务层有策略地保留可复用前缀, 无需因临时内存压力而朴素地逐出它们。其次, 了解请求在工作流中的大致位置后, 调度器能够估计剩余执行时间与依赖关系, 实现感知关键路径的请求调度。最后, 这种结构感知使系统能够预见即将到来的阶段转换和扇出模式, 主动缓解突发导致的排队。

在生产环境中获取这种结构预知信息十分可行。AutoGen、LangGraph 等框架虽然理论上支持完全动态、无约

表 1: 编码助手中各智能体角色的输出长度统计。

智能体角色	平均输出长度	CV
Explorer (探索者)	1924	0.45
Chronicler (记录者)	912	0.26
Architect (架构师)	1194	0.22
Planner (规划者)	60	0.15
Engineer (工程师)	3152	0.45
Reviewer (审查者)	2620	0.18
Verifier (验证者)	65	0.16

束的智能体群聊，但实际部署很少采用这种方式。由于无约束的 LLM 容易出现幻觉和任务偏离，企业应用通常设置更严格的路由约束及运行保护规则。例如，我们的内部编码助手通过硬编码转移规则约束执行，仅在边缘情况采用基于 LLM 的群聊选择器作为后备。因此，生产工作流趋于稳定，沿高概率执行路径运行，迭代循环受到良好约束，智能体角色之间的转移概率高度集中。

机会 #2: 正在处理请求的输出长度。预测活跃请求的输出长度有两个关键运行优势。首先，它支持感知资源的调度，缓解严重内存争用和负载失衡。服务层按预测资源消耗平衡系统，而不是朴素地按请求数平衡，从而避免触发严重 HoL 阻塞的局部内存压力。其次，估计活跃请求的解码时长，可以细粒度地确定它何时到达工作流中的下一个智能体。服务系统据此能够恰逢其时地获取并预热后续智能体的 KV 缓存。

这种可预测性建立在工作流内严格的分工之上。每个智能体的任务范围具体且狭窄，其生成行为自然收敛为稳定、可识别的特征。如表 1 所示，Chronicler、Planner、Reviewer 和 Verifier 可靠地产生相对一致的长度，变异系数 (CV) 较低。Explorer 和 Engineer 因交互特性而具有较高输出长度方差，但我们观察到连续调用之间存在很强相关性。因此，系统通过观察智能体的语义角色和当前上下文，可以大致约束请求的计算/内存占用。

机会 #3: 未来提示的组成与来源。预测即将到来的提示结构，使服务系统能够主动掩盖昂贵的预填充时延。专门化智能体依赖不同的系统提示，无法原生共享前缀缓存；当工作流从一个智能体转到另一个时，新请求通常触发一次完整且昂贵的预填充计算。然而，如果系统预测到下一条提示，就能在实际请求发出之前，提前为后续智能体主动计算并准备 KV 缓存。这种主动的“预先预填充”能够有效地将预填充瓶颈从关键路径中移除。

多智能体框架的机制自然暴露了后续请求的确切内容。后续提示并非黑盒，而是可靠地通过拼接已知静态内容 (e.g. 智能体的具体系统指令)，以及工作流中此前已执行的依赖智能体的显式输入和生成输出构造。服务层已经跟踪工作流图并持有前面步骤的输出，因此具备所有必要信息，可以提前准确组装并预计算下一条提示。

要点。多智能体负载的程序性质带来很强的结构、时间和语义可预测性。但这些宝贵的先验信息在当今无状态的应用—系统接口中完全丢失。为解决由此引起的缓存抖动、负载失衡和突发排队等级联问题，必须赋予服务层某种预知能力。

3 Pythia 概览

图 6 展示 Pythia 的系统概览。

生成预测信息。Pythia 的入口是兼容 OpenAI 的 API (§4.1)，扩展后支持轻量元数据注解。这些简单注解用于在 LangGraph、AutoGen 等常用框架中直接识别不同工作流及具体智能体角色。这一最小 API 扩展对最终用户完全透明，无需改变应用逻辑。

除了收集多智能体工作流的结构信息，性能剖析器 (§4.1) 还作为 Pythia 的分析核心，在关键路径之外异步运行。它持续摄取带注解的请求/响应日志，提取并维护整体控制流图以及不同智能体的程序特征。当新请求到达时，它利用传入元数据查询历史特征，进一步为请求添加三类关键执行估计，供下游模块使用。

使用预测信息。Pythia 在服务流水线的三个位置使用上述预测信息：各节点（智能体）的前缀缓存管理、全局请求调度，以及各节点的模型扩缩容，分别对应现有技术面临的三个主要挑战 (§2)。

节点级缓存管理器 (§4.2.1) 通过将被动 LRU 逐出改为主动的类 Belady 策略，缓解前缀抖动。它利用提取的工作流信息准确预取共享上下文，并立即丢弃不再需要的短暂 Token。此外，它预测后续请求的确切提示组成，为最可能的下一智能体异步预热缓存，在后续请求尚未到达前有效掩盖预填充时延。

集群级请求调度器 (§4.2.2) 利用提取的程序特征，执行感知资源的负载均衡和精确请求优先级分配。它依据预测输出长度，将异构负载均匀分配到数据并行副本。此外，这种感知性能特征的方法建立更好的执行优先级，有效缓解导致请求抢占的局部内存争用和 HoL 阻塞。

各节点的模型自动扩缩容器 (§4.2.3) 通过预见负载变化来管理整个集群的容量，而不只是事后响应。它结合提取的工作流模式评估当前活跃请求的状态，预测即将到来的扇出、阶段转换等结构性变化。借助这种预知信息，系统能够在突发到达前主动增加专用模型容量，并在不再需要时缩减容量，有效缓解突发导致的排队。

我们在 §5 讨论 Pythia 如何支持多种不同工作流、响应模式变化并处理对抗行为。

4 Pythia 的设计

4.1 产生预测信息

为了预测服务各阶段的资源用量，Pythia 首先提取入站请求的语义上下文。我们的目标是在不给应用开发者增加沉重集成负担、也不破坏应用逻辑与服务基础设施分离原则的前提下做到这一点。为此，我们引入一个最小且透明的 API 扩展，并配合异步后台分析器，利用经典程序分析技术对工作流执行建模。

API 抽象。标准推理 API 将请求彼此隔离，去掉了它们所属应用的整体上下文。为了弥补这一缺口，Pythia 使用 OpenAI 兼容端点原生支持的 `extra_body` 参数。智能体框架发出请求时，会注入轻量的 `app_metadata` 载荷，标识执行上下文。如代码清单 1 所示，框架只需提供三个明确的标识符：

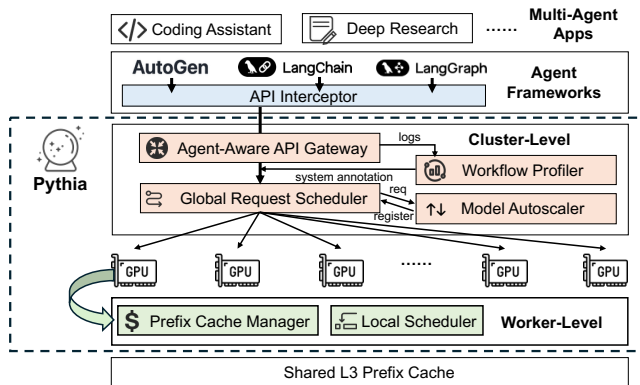


图 6: Pythia 概览。

1. workflow_type_id 唯一标识一个应用 (e.g., coding_assistant)，使系统能够将请求映射到特定的结构分析结果。
2. workflow_id 唯一标识一个会话，使服务层能够跟踪某个执行实例正在延续的来源关系与上下文历史。
3. agent_id 唯一标识工作流中的当前节点或角色 (e.g., engineer)，明确指出当前执行的是哪个子任务。

我们将 API 扩展限制在这三个标识符，而不要求应用传入显式定义的依赖图或调度提示。预先要求完整的工作流图和智能体特征并不现实，因为这些信息往往在运行时动态确定。反过来，要求应用开发者手工给请求标记系统层面的提示 (e.g., “缓存此前缀”)，会破坏关注点分离原则，让他们承担基础设施资源管理的负担。只传递工作流和智能体的身份，Pythia 就能轻松地将请求与历史模式关联，将预测和资源分配的责任完全交给底层服务系统。

这一最小抽象便于部署。对于构建 LangGraph、AutoGen 等编排层的框架开发者，集成 Pythia 无需根本改变架构。他们只需配置一个全局 API 拦截器，在请求发出前自动将当前会话和智能体变量附加到 extra_body 载荷中。因此，基于这些框架构建应用的开发者能够透明地获得先进系统优化的收益，无需改动应用逻辑或编写感知硬件的代码。

通过正则表达式对执行轨迹建模。这些带注释的请求进入系统后，工作流分析器异步读取日志，重建应用的全局行为。受到动态程序分析 [24] 和程序合成 [6] 的启发，我们把智能体调用序列视为以智能体角色为字母表的执行轨迹。分析器挖掘历史轨迹，构建一个概率有限自动机 (PFA)。

分析器并非只预测紧接着的单个智能体；它将 PFA 综合为一个正则表达式 (regex)，投影出整个预期执行图。考虑一个复杂的编码工作流：planner 将任务委派给多个并行的 explorers，随后 engineer 迭代改进代码，再调用 reviewer，并可能进入返工循环。分析器不会把它表示为 (engineer | reviewer)* 这样的无限、不可预测的循环，而是将执行限制在高概率的结构边界内。如代码清单 1 的 predicted_path_regex 字段所示，系统捕获了并行扇出

代码清单 1: API 载荷：框架通过 app_metadata 注入轻量的身份元数据；工作流分析器拦截它，并在路由前通过同一 extra_body 块中的 sys_annotations 注入可用于执行优化的预测。

```
{
  "model": "meta-llama/Llama-3.1-8B-Instruct",
  "messages": [{"role": "user", "content": "..."}],
  "extra_body": {
    "app_metadata": {
      "workflow_type_id": "coding_assistant",
      "workflow_id": "<session_id>",
      "agent_id": "engineer"
    },
    // 由工作流分析器在内部注入
    "sys_annotations": {
      "predicted_output_len": [1000, 1300],
      "predicted_path_regex": "planner -> (explorer)^(1|3,4) ->
        (engineer)^(3,6) -> reviewer -> (engineer)^(2-4) ->
        reviewer)? -> verifier -> terminal",
      "prompt_composition": {"engineer": "你是一位乐于助人的工程师。
        (explorer)^(1|3,4)、有界的顺序自我反思 ((engineer)^(3,6),
        以及可选的单次重试后向边 ((...)?)。
        此外，分析器将接下来提示的结构建模为确定性模板。
        prompt_composition 中的每个占位符都充当严格的内存
        指针，由三个精确参数组成：(1) 所依赖智能体的历史
        request_id (e.g., req_1)；(2) 指明从该智能体的输入
        request 还是生成的 response 中提取内容的标志；(3) 待
        取回内容的具体词元索引 range。代码清单 1 展示了这
        一模板如何捕获智能体拼接各组成部分的过程。
        面向占主导的执行分位范围优化。将程序分析应用于智
        能体工作流时，一个关键认识是：为每一种可能的执行
        边界情况建模反而适得其反。由于 LLM 固有的随机性，
        工作流偶尔会出现异常转移、进入混乱的错误恢复循环，
        或触发失控的词元生成。试图完整考虑这些离群情况，
        会导致无限递归的表示和过度预留的内存。
        因此，Pythia 有意针对控制流和资源用量中占主导的
        执行分位范围进行优化。它通过轨迹过滤剪除低概率转
        移 (e.g., 出现比例低于 5% 的转移)，得到能够紧密刻画
        常见执行图的路径表达式。同样，分析器不预测精确输
        出长度，也不按最坏情况的上下文窗口预留资源，而是
        为每个智能体推导高置信度区间 (e.g., 历史词元用量的
        第 99 百分位数)。这一经过过滤的概率视图符合实际部
        署：它反映结构化路由与护栏对 LLM 稳定路径的约束，
        同时限制内存用量，避免对罕见长尾行为反应过度。
        可直接用于优化的请求注释。推理时，分析器运行在网
        关处。请求到达后，它提取 app_metadata，查询经过过滤
        的模型以取得预期输出长度、确定描述后续 workflows 的
        正则表达式，并约束下一条提示的模板。随后，它将这
        些信息注入 extra_body 载荷的 sys_annotations 字段。一
        次拦截便能将原本不透明的请求转变为信息充分的数据
        结构，为下游组件提供精确的执行计划和内存提示，以
        进行主动缓存、全局调度和集群自动扩缩容。
        5
```

算法 1 推测式缓存管理。

输入：当前请求 R_{curr} ，分层缓存 C ，GPU 计算流 S_{bg}

- 1: $regex \leftarrow R_{curr}.predicted_path_regex$
- 2: $future_nodes \leftarrow ParseRegex(regex)$
- 3: $prompt_dict \leftarrow R_{curr}.prompt_composition$
- 4:
- 5: 过程 ONREQUESTCOMPLETE
- 6: 对于每个缓存块 $b \in C$ 执行
- 7: 若 $b.lineage \notin future_nodes$ 则
- 8: Free(b) ▷ 词元已无后续用途，立即丢弃
- 9: 否则若 $b.lineage \in future_nodes$ 则
- 10: RetainInHierarchy(b)
- 11:
- 12: 过程 ONPREFETCHREQUESTED
- 13: 对于每个 $(agent, template) \in prompt_dict$ 执行
- 14: // $prompt_dict$ 只有一个条目
- 15: $prompt_next \leftarrow Assemble(template, History())$
- 16: 若 ExistsInCache($C, prompt_next$) 则
- 17: PromoteToHost($prompt_next$)
- 18: 否则若 IsIdle(GPU) 则
- 19: Enqueue($S_{bg}, ForwardPass(prompt_next)$)

4.2 使用预测信息

4.2.1 推测式缓存管理。 借助分析器产生的工作流图和提示模板，Pythia 使用推测式缓存管理器，缓解多智能体工作负载固有的前缀缓存抖动与缓存未命中。利用嵌入请求 $sys_annotations$ 中的未来执行图，缓存管理器把内存管理从被动猜测转变为类似 Belady 算法的主动策略。如算法 1 所示，这通过结合提前缓存驱逐和前向缓存预置实现。

提前缓存驱逐（第 5–10 行）。 每条请求完成时（OnRequestComplete），缓存管理器主动裁剪 KV 缓存，以防止内存膨胀并保护关键上下文。解析 $predicted_path_regex$ 后，它生成一个确定的未来智能体节点集合，然后对照该集合评估缓存中每个块的来源关系。不再被访问的临时步骤所对应的块被归类为无后续用途的词元，并立即释放，绕过标准 LRU 策略延迟回收的过程。相反，若某个块的来源出现在未来路径中，它会被明确标记为保留。管理器并非仅将这些块留在本地工作节点，而是积极地把它们写入所有推理引擎均可访问的共享 L3 存储层。通过主动将与未来执行相关的上下文持久化到全局缓存空间，Pythia 将缓存驻留位置与工作节点亲和性解耦。这简化了集群的全局路由策略，因为多智能体工作流中的后续请求可以调度到任意可用工作节点，而不会遭遇完全缓存未命中。

前向缓存预置（第 12–19 行）。 为了缓解智能体之间切换时的预填充延迟，管理器主动准备预期下一步的精确上下文（OnPrefetchRequested）。依据请求注释提供的 $prompt_composition$ 模板，管理器从执行 History() 中注入相关输入和输出，组装后续提示。目标提示组装完成后，管理器确保对应前缀缓存尽可能靠近计算单元。如果这一提示的缓存已存在于更深层的共享 L3 存储中，管理器就把它提升至 L2 主机 DRAM（PromoteToHost）。

算法 2 基于统计容量的路由。

输入：请求 R ，候选节点 N ，目标 OOM 阈值 ϵ

- 1:
- 2: 过程 LROUTE(R, N, ϵ)
- 3: $N_{capable} \leftarrow \emptyset, target \leftarrow null, max_headroom \leftarrow 0$
- 4: 对于每个 $n \in N$ 执行
- 5: $P_{oom} \leftarrow Prob(\sum_{r \in n.active \cup \{R\}} r.len > n.capacity)$
- 6: 若 $P_{oom} \leq \epsilon$ 则
- 7: $N_{capable} \leftarrow N_{capable} \cup \{n\}$
- 8: $headroom \leftarrow n.capacity - ExpectedSize(n)$
- 9: 若 $headroom > max_headroom$ 则
- 10: $max_headroom \leftarrow headroom$
- 11: $target \leftarrow n$
- 12: 返回 $target$ ▷ 容量相同时以缓存亲和性决胜

关键在于，系统有意将预取上下文保存在 L2 主机内存中，而不立即推入资源高度受限的 GPU HBM。这一设计考虑了两个现实因素：第一，分析器对下一智能体的预测具有推测性，可能出错；第二，当前请求可能继续生成词元较长时间，激进地搬移推测数据会过早污染有限的 GPU 内存。前向预置类似于物流中的提前布点：它把所需内存块保存在限制较少、距离 GPU 仅一跳的位置，实际请求到达时即可按需立即经 PCIe 传输。

但如果组装后的提示在缓存层级的任何位置都不存在，管理器会尝试从头生成它。缓存管理器择机使用 GPU 计算单元；检测到 GPU 空闲周期时，它把异步前向计算加入队列，在后台预先计算前缀缓存。结合主机级预置与后台预填充，系统最大化了框架发出后续推理请求之前前缀已经预热的概率。

4.2.2 前瞻请求调度。 分析器提取多智能体请求元数据，缓存管理器管理上下文，Pythia 则通过两级调度架构优化整个集群的执行：集群级的统计容量路由器，以及工作节点级的图驱动本地调度器。

统计容量路由。 在集群入口处，全局路由器将入站请求分配到具体工作节点，如算法 2 所示。由于 LLM 输出生成固有的随机性，依赖静态最大词元长度会造成严重的内存利用不足，而依赖简单均值则会因内存不足（OOM）触发请求抢占。Pythia 利用分析器提取的高置信度区间解决这一问题。

对于每个请求 r_i ，分析器提供一个由高置信度区间得到的预期长度上界 u_i （e.g., 第 99 百分位数，对应错误概率 $\alpha_i = 0.01$ ）。这保证了实际长度 L_i 超过该上界的概率为 $P(L_i > u_i) \leq \alpha_i$ 。为了避免对底层统计分布作任何假设，Pythia 使用不依赖分布的严格容量预留。如果所有活跃请求 A 与新请求 R 的置信上界之和可以放入内存，则总 KV 容量为 C 的候选节点被视为在结构上具备容纳能力：

$$\sum_{r_i \in A \cup \{R\}} u_i \leq C$$

满足这一预留条件时，只有实际词元生成违反某些单请求上界才可能发生 OOM。应用联合上界（Boole 不等式），路由器计算 OOM 事件最坏情况下的联合概率 P_{oom} ；该

算法 3 图驱动的全局优先级分配。

输入： 集群副本队列 R_{queues} ， 请求 r

```

1:
2: 过程 DOWNSTREAMIDLERISK( $R_{queues}, r$ )
3:    $risk\_score \leftarrow 0$ 
4:    $future\_agents \leftarrow \text{GetFutureAgents}(r.regex)$ 
5:   对于 每个  $a \in future\_agents$  执行
6:     若  $R_{queues}[a.model] < \text{IDLE\_THRESHOLD}$  则
7:        $E[D_a] \leftarrow \text{ExpectedDist}(r.agent, a)$ 
8:        $risk\_score \leftarrow risk\_score + \left(\frac{1}{E[D_m]}\right)$ 
9:   返回  $risk\_score$ 
10:
11: 过程 SETPRIORITY( $r, R_{queues}$ )
12:    $E[D_{remain}] \leftarrow \text{ExpectedRemainingDistance}(r.regex)$ 
13:    $S_{completion} \leftarrow \frac{1}{E[D_{remain}]}$ 
14:    $S_{unblock} \leftarrow \text{DownstreamIdleRisk}(R_{queues}, r)$ 
15:    $r.base\_priority \leftarrow (\omega_1 \times S_{completion}) + (\omega_2 \times S_{unblock})$ 

```

概率严格受各请求错误概率之和约束：

$$P_{oom} \leq \sum_{r_i \in AU\{R\}} P(L_i > u_i) \leq \sum_{r_i \in AU\{R\}} \alpha_i$$

路由器要求这一不依赖分布的失败概率不超过严格的、由用户设定的安全阈值 ($P_{oom} \leq \epsilon$)。在数学上安全的节点中，它把请求路由到预期余量最大的节点 ($C - \sum u_i$)。前向缓存预置 (§4.2.1) 虽然不属于主要容量约束的严格要求，路由器仍将其用作确定性的同分决胜规则。多个节点提供安全容量配置时，路由器优先选择已经持有所需 L2 主机缓存的节点，将严格的统计负载均衡与即时缓存命中结合起来。

图驱动的全局优先级分配。在多个智能体共享底层模型副本的多智能体环境中，标准 FCFS 调度经常造成整个集群的饥饿。它忽略工作流依赖，使新产生的长时间运行智能体阻塞最后阶段的智能体，严重延迟最后阶段的完成。为解决此问题，Pythia 实现了全局仲裁策略（算法 3 中的 SetPriority）。请求路由到工作节点之前，全局调度器为其分配静态的 $base_priority$ 分数。

这一分数明确平衡两个集群级执行目标。首先，通过优先处理最接近工作流终点的请求，加速作业完成。由于工作流路径由概率正则表达式而非确定性图定义，调度器计算到终止节点的剩余距离的数学期望 ($E[D_{remain}]$)。优先级分数与该期望成反比，确保统计上更接近完成的智能体 (e.g., 最终验证者) 天然优先于早期阶段智能体，从而尽快将已完成作业移出系统。

其次，调度器主动监测集群所有模型副本的实时队列深度 (DownstreamIdleRisk)。如果服务未来智能体的模型副本面临空闲风险，就提高最终会解除其依赖的任意上游请求的优先级。关键是，该提升幅度按当前智能体与饥饿模型之间的期望距离 ($E[D_m]$) 的倒数加权。只需一步便可给空闲 GPU 提供工作的请求，其优先级显著上升；距离十步的请求仅获得微小提升。这确保任何专用推理工作节点都不会因等待停滞的上游依赖而处于空闲。

周期性的本地批次构建。请求路由到工作节点后，执行由迭代级派发控制。每个调度窗口开始时，本地工作节点将按累计等待时间缩放的老化因子加到全局 $base_priority$ 上，动态重新计算所有请求的有效优先级。随后对请求池排序，派发内存限制内能够容纳的最高优先级子集。这使早期阶段智能体在数学上受到保护，避免永久饥饿，同时保留全局图希望实现的执行次序。

感知优先级的抢占恢复。偶发的长尾生成导致工作节点耗尽内存时，Pythia 替代标准 FCFS 驱逐策略。本地调度器不会盲目地将被驱逐请求追加到队尾，而是专门选择动态优先级最低的活跃请求 (e.g., 早期阶段智能体) 作为抢占对象。该请求被暂停并重新插入本地队列。由于每次迭代前都严格重新排序队列，局部内存峰值不会扰动关键路径上的执行，从而实现较高的有效资源利用率，并最小化工作流级等待时间。

4.2.3 随阶段适配的自动扩缩容。前瞻请求调度 (§4.2.2) 能够缓解负载不均衡，却无法创造新容量。智能体应用经常因同步触发 (e.g., cron 定时作业) 或大规模结构性扇出而表现出极强的突发性。在这些剧烈变化期间，仅靠调度无法防止队列饱和；集群必须实际扩展模型副本数量。

现有无服务器 LLM 方案依赖反应式自动扩缩容，在多智能体执行常见的迅猛、瞬时阶段变化期间失效。由于将大型模型权重加载到 GPU 内存需要数十秒，反应式资源供应不仅太慢，还可能主动损害系统。在资源受限的集群中，启动新模型往往会迫使系统盲目驱逐现有模型。等到新副本终于就绪，短暂突发通常已结束，系统留下空闲模型，同时破坏了其他关键模型的缓存，触发一连串冷启动。

Pythia 利用工作流分析器提取的正则路径表达式，引入预测式控制平面来克服这一问题。它预测即将发生的突发，并在流量转移前主动供应副本，而不是等到队列积压才反应，从而完全避免反应式资源抖动循环。

针对结构变化主动扩容。Pythia 不等到队列激增，而是持续将所有活跃请求的状态与各自工作流图对照，预测即将到来的需求（算法 4 中的 EstimateImminentDemand）。自动扩缩容器将执行图向前投影到指定步数范围 (H)，计算哪些模型将在近期被大量使用。

例如，集群当前有五十个活跃请求正在执行 planner 阶段，而底层正则表达式规定转移为 $planner \rightarrow (explorer)^{110}$ ，则自动扩缩容器可从数学上确定近期将有 500 条请求到达 explorer 模型。在 planner 智能体尚未完成输出生成时，Pythia 就主动在后台供应更多 explorer 模型副本。框架真正发出大规模扇出突发时，新 GPU 节点已完成预热并就绪，有效缓解突发造成的排队，同时隐藏冷启动延迟。

预测式缩容与资源回收。在资源受限的集群中，扩展一个专用模型通常要求驱逐另一个模型。标准系统依赖静态保活超时 (e.g., 等待 5 分钟再关闭空闲模型)，在关键阶段变化时仍占用宝贵 GPU 内存。Pythia 通过预测式缩容加速这一回收过程。

由于自动扩缩容器掌握活跃工作负载的完整结构图，它能够识别近期不会被调用的模型。这包括服务已完全

算法 4 随阶段适配的自动扩缩容。

输入： 活跃请求 A ，当前副本 R_{curr} ，前瞻范围 H

```
1:
2: 过程 ESTIMATEIMMINENTDEMAND( $A, H$ )
3:    $D \leftarrow \text{InitializeZeroMap}()$ 
4:   对于 每个  $r \in A$  执行
5:      $\text{imminent\_agents} \leftarrow \text{ProjectGraph}(r.\text{regex}, H)$ 
6:     对于 每个  $a \in \text{imminent\_agents}$  执行
7:        $D[a.\text{model}] += \text{EstimatedLoad}(a)$ 
8:   返回  $D$ 
9:
10: 过程 AUTOSCALECLUSTER
11:    $D \leftarrow \text{EstimateImminentDemand}(A, H)$ 
12:    $R' \leftarrow \text{InitializeMap}()$ 
13:   对于 每个  $m \in \text{RegisteredModels}()$  执行
14:      $R'[m] \leftarrow \text{EstimateReplicas}(D[m])$ 
15:   对于 每个  $m \in \text{RegisteredModels}()$  执行
16:     若  $R'[m] < R[m]$  则
17:        $\text{ScaleDownOnIdle}(m, R[m] - R'[m])$ 
18:   对于 每个  $m \in \text{RegisteredModels}()$  执行
19:     若  $R'[m] > R[m]$  则
20:        $\text{ScaleUp}(m, R'[m] - R[m])$ 
```

成为过去步骤的智能体的模型 (e.g., 所有工作流都已进入 explorer 阶段后的 planner)，以及服务拓扑上距离过远、没有必要立即预热的智能体的模型 (e.g., 距当前执行前沿很多步的 final_verifier)。为了最大化安全缩容的机会，自动扩缩容器直接与全局调度器协调。一旦某模型被判定为在当前前瞻范围内已经不再需要，自动扩缩容器就通知调度器停止将新请求负载均衡到这些特定副本。主动排除路由使副本能够快速排空队列。目标副本的即时队列一旦为空，Pythia 就强制撤销该副本，无需等待任意设定的超时。这种积极的、图驱动的回收确保集群资源持续释放并重新分配，以支持当前执行阶段所需的模型。

5 讨论

支持大量不同的智能体工作流。目前，Pythia 主要面向运行少量特定智能体应用的环境。系统扩展为并发服务种类极多的不同工作流时，Pythia 仍能保持最小化全局平均 JCT 这一核心优势。但在如此高度异构的环境中，依赖当前相对简单的调度启发式，可能为了最大化整体吞吐量而无意中牺牲某些单独工作流的性能。为了有效平衡集群级资源效率与其他用户要求 (e.g., 逐工作流 SLO 或公平性保证)，Pythia 的前瞻调度器可以自然扩展。将逐工作流优先级权重与明确的延迟目标直接纳入调度算法，系统就能作出更细致、感知 SLO 的分配决策，既保护关键工作流，又保留整体优化。

冷启动与工作流漂移。Pythia 依赖历史轨迹，这会给初次部署（冷启动）和应用更新（工作流漂移）带来挑战。为防止不足或陈旧的数据触发不安全路由，Pythia 可以将显式 API 版本管理与“影子分析”阶段结合。当 app_metadata 表明出现新的工作流版本时，网关暂时通过标准反应式回退路径路由其流量，同时在后台对其进

行分析。建立安全的统计置信区间后，系统再将该工作流无缝提升到主动执行层，从而保证生命周期转换中的稳定性。

开放式与对抗性工作流。Pythia 依赖企业应用通常具备的结构可预测性：这些应用设置路由护栏，并假设输入可信。在完全不受约束或对抗性场景下，执行图退化为不可预测的网状结构，使主动优化失效。为适应这些环境，未来部署可以采用混合架构：由 Pythia 服务经过验证、可预测的工作流，把高度偏离或对抗性请求转交到隔离的反应式调度回退环境。

6 实现

我们在 SGLang 上实现了 Pythia。工作流分析器与全局请求调度器替代默认 SGLang 路由器，拦截标准 OpenAI 兼容请求以提取 app_metadata，注入 sys_annotations，并在路由前评估统计容量界限 (§4.2.2)。在引擎层面，我们将推测式缓存管理器 (§4.2.1) 集成到 SGLang 的分层缓存中。此外，我们以图驱动的迭代派发器替代 SGLang 默认调度器，确保每次迭代开始时都严格按照动态 priority 分数对请求排序。最后，随阶段适配的自动扩缩容器 (§4.2.3) 作为控制平面的周期性守护进程运行，根据投影得到的需求，在 GPU 集群中创建和撤销 SGLang 模型副本。

7 评估

实验平台。我们在一台配备两颗 AMD EPYC 7J13 CPU、1.7TB DRAM 和八张 NVIDIA A100 GPU (每张 80GB HBM，并通过 NVLink 互联) 的服务器上评估了 Pythia。

基线。我们将 Pythia 与一组全面的先进 LLM 服务系统和感知智能体的框架比较：

- **vLLM [1]:** 部署 vLLM (v0.17.1)，配合 vLLM Production Stack [25] (v0.1.10) 路由器。
- **SGLang [2]:** 使用 SGLang (v0.5.9)，配合其网关。
- **Autellix [26]:** 专门面向智能体的 LLM 服务系统，在路由器和推理引擎之间提供集成的调度优化。由于 Autellix 尚未开源，我们在 vLLM (v0.17.1) 上实现了相同的路由和调度策略。
- **Continuum [23]:** 针对 ReAct [27] 智能体优化的单节点服务系统；为公平比较，我们将它与 vLLM Production Stack 路由器 (v0.1.10) 一起使用。我们只允许可能产生缓存命中的智能体固定其缓存。
- **ThunderAgent [22]:** 面向智能体应用、感知程序的全局 LLM 请求调度器，底层推理引擎采用 SGLang (v0.5.9)。ThunderAgent 最初针对单模型设计，因此我们将其单队列设计改为每个模型一个队列；并将调度间隔从默认 5 秒改为 0.5 秒，以减少不必要的排队延迟。

所有系统均使用 Mooncake [15] 作为 L3 前缀缓存；基于 vLLM 的基线则通过 LMCACHE [16] 集成。

工作负载。我们采用移植到 AutoGen [7] 框架的两个代表性智能体工作流评估 Pythia：一个内部专有编码助手，

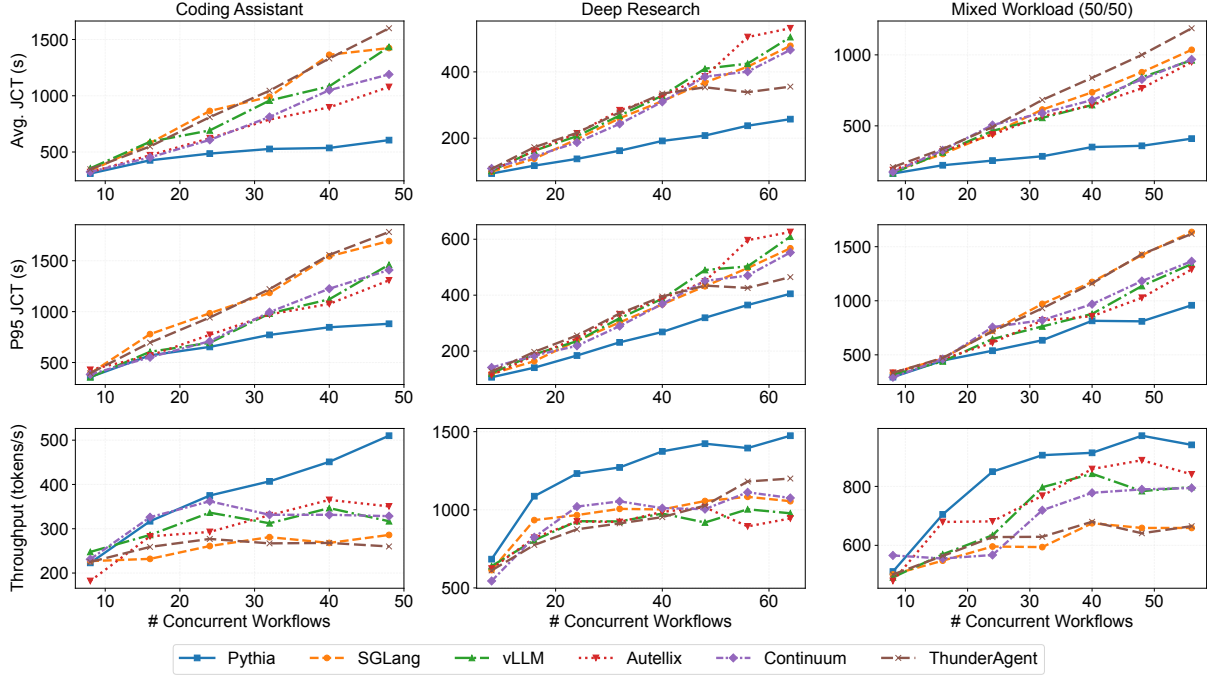


图 7: 端到端实验。

以及一个深度研究智能体 [28]。使用这些应用, 我们执行两个具有挑战性的基准: **SWE-bench Pro** [29], 一个反映复杂生产模式的严格编码基准; 以及 **Deep Research Bench** [30], 用于评估博士级研究任务。

评估采用这些应用的真实轨迹, 轨迹覆盖分配给不同智能体角色的多种前沿模型, 包括 Anthropic、OpenAI 和 Google 的模型。为了高保真地模拟智能体行为, 轨迹采集和评估实时接入真实执行环境与生产级搜索 API。

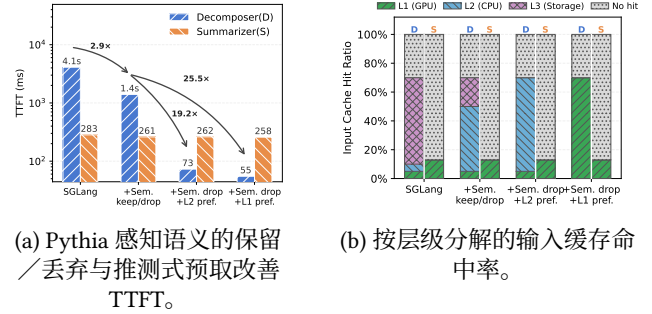
为便于在实验平台部署, 我们将这些前沿模型映射为 Qwen 和 Llama 系列中较小规模的模型 (3B–14B), 采用 1 至 4 的张量并行 (TP) 度。

7.1 端到端结果

我们首先在不同并发 workflows 数量下, 将 Pythia 与全部五个基线系统比较。比较指标为平均 JCT、P95 JCT 和吞吐量 (每秒生成的输出词元数)。

平均 JCT。 在所有工作负载中, Pythia 在峰值并发下将平均 JCT 改善 1.38–2.9 $\times$ 。低负载时, 收益主要来自推测式缓存管理对预填充延迟的隐藏。负载密度增加时, Pythia 的图驱动全局仲裁避免了困扰基线的严重队头 (HoL) 阻塞, 异构混合工作负载下尤其明显。部分感知智能体的系统 (Continuum、Autellix) 通常优于不感知智能体的引擎, ThunderAgent 在高负载深度研究场景下也表现出一定韧性, 而 Pythia 在所有场景中保持稳定性能。

P95 JCT。 近似 SRTF 调度通常会使得早期阶段请求饥饿, 从而恶化尾延迟, 但 Pythia 克服了这一权衡, 在高并发下将 P95 JCT 改善 1.15–2.02 $\times$ 。Pythia 并非仅偏好短输出, 而是动态提升能够解除下游模型依赖的上游请求的优先级。配合优先级老化, 这一整体策略避免复杂多步工作



(a) Pythia 感知语义的保留 / 丢弃与推测式预取改善 TTFT。

(b) 按层级分解的输入缓存命中率。

图 8: 感知工作流的推测式前缀缓存管理改善 TTFT 延迟及来自较低缓存层级的前缀缓存命中率。

流长期滞留队列, 即使极端负载下也能确保高度可预测的尾部性能。

吞吐量。 在所有评估的工作负载中, Pythia 将吞吐量提高 1.12–1.96 $\times$ 。这一收益来自两个互补机制: 统计容量路由器利用输出长度预测均衡分配负载, 尽量减少局内存竞争; 图驱动调度器优先处理关键路径任务, 防止下游模型空闲, 最终最大化集群的总体利用率。

7.2 性能分析

缓存效率。 我们以深度研究工作流说明 Pythia 推测式缓存管理的有效性。深度研究工作流可以重复数轮, 其中分解器 (Decomposer) 在轮次之间共享上下文, 而汇总器 (Summarizer) 接收研究员 (Researchers) 提供的大量新上下文。如图 8 所示, 协同实施主动保留 / 丢弃和及时预取后, 分解器复用 L1/L2 缓存中的词元, 将 TTFT 改善了 25.5 $\times$ 。

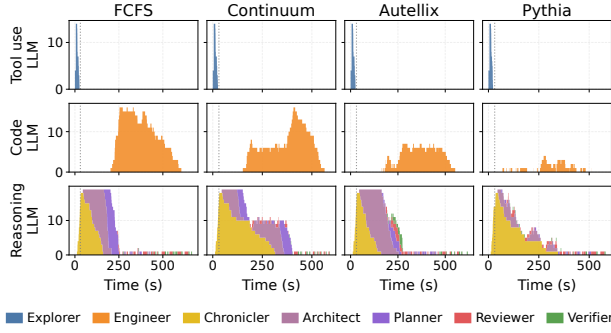


图 9: 编码助手采用不同调度策略时的 GPU 队列动态。Pythia 实现了更低的排队延迟与更均衡的 GPU 利用率。

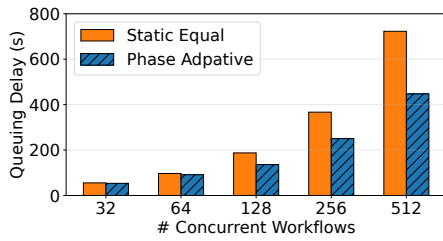


图 10: 不同扩缩容策略下的排队延迟。

不同调度算法的比较。为分析图驱动全局优先级分配的有效性，我们分解服务编码助手工作流突发时的排队动态。图 9 表明，三个基线在代码 LLM 和共享推理 LLM 上均形成明显更大的队列。FCFS 盲目服务先到请求，使上游推理请求阻塞较后阶段的审查和验证，延迟完成，最终导致代码 LLM 积压。Continuum 同样在整个工作流中保留作业级到达次序，减慢已接近完成的请求的排空速度。Autellix 优于 FCFS，但因不了解工作流位置和下游瓶颈，仍接纳过多上游请求，留下显著队列积压。相反，Pythia 利用工作流结构和剩余工作量，优先处理对完成至关重要的请求，并在瓶颈出现时限制上游注入，从而防止排队浪潮在 GPU 之间传播，带来更低 JCT 和更高吞吐量。

自动扩缩容的有效性。图 10 在改变并发工作流数量时，将 Pythia 与静态资源分配比较。Pythia 通过模型自动扩缩容主动重新均衡资源，相对静态模型放置将排队延迟改善 1.49 $\times$ 。关键优势在于，Pythia 预测工作流各阶段的负载模式，为正经历突发请求到达的阶段分配额外模型实例。静态放置无法响应这些变化的瓶颈。并发增加时，更多工作流同时遍历不同阶段，放大突发性，并在固定资源供应下造成长请求队列。

7.3 消融研究

对负载突发性的敏感性。智能体工作流经常成批到达，因此底层服务系统应能充分处理突发负载。图 11 展示到达间隔具有不同变异系数 CV 时的平均 JCT 和吞吐量 (0: 稳定; 1: 类似泊松; >1: 突发)。Pythia 的性能相对一致，而各基线出现波动。

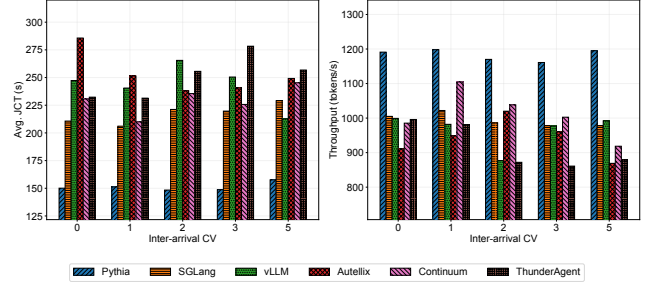
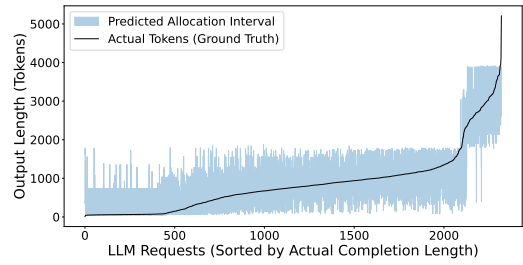
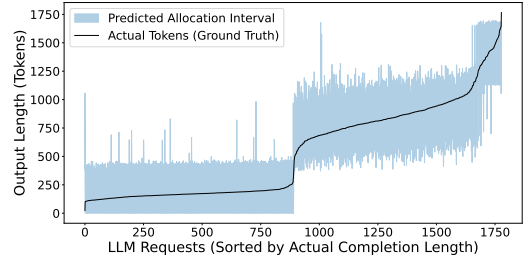


图 11: 对负载突发性的敏感性。



(a) 编码助手。



(b) 深度研究。

图 12: 预测输出长度与实际长度的比较。

预测器准确性。图 12 将测试集上的预测输出长度区间与真实长度比较。对于编码助手和深度研究，除少量离群点外，Pythia 的预测器能为大多数数据点提供界限良好的预测。Pythia 还能准确提供这些应用的工作流程图，因为其模式总体稳定。

8 相关工作

LLM 服务系统。现代服务引擎在多个层面进行了高度优化。在调度方面，Orca [11] 引入迭代级调度来处理不同请求长度，FastServe [31] 使用抢占缓解队头阻塞。为了管理异构执行阶段，系统探索了阶段拆分 [12, 32–34] (e.g., 预填充与解码拆分)，或阶段共置 [3, 35] (e.g., 分块预填充)。内存管理同样关键：PagedAttention [1] 使用操作系统虚拟内存抽象减少 GPU 内存碎片，其他工作则通过缓存和内存层级加速预填充阶段 [2, 15, 16, 36, 37]。在算子层面，FlashInfer [38]、FlashAttention [39] 等库提供基础的高性能注意力核。最后，多 LLM 服务系统聚焦于空间与时间复用，最大化共享硬件上共置模型的密

度 [4, 40–43]。与优化智能体工作流端到端性能的 Pythia 不同, 这些系统主要针对细粒度的请求级和词元级指标。

面向智能体 AI 的系统优化。近期工作已经认识到, 服务智能体 AI 应用会带来超越传统 LLM 推理的新系统挑战, 因此需要专门运行时和服务框架中的针对性优化。Pancanke [44] 设计了高效的多级智能体记忆系统, DualPath [21] 优化了智能体工作负载远程前缀缓存的网络瓶颈。ThunderAgent [22]、Ayo [45] 和 Autellix [26] 等系统着重改进长时程复杂智能体流水线的请求编排, 而 KVFlow [46] 与 Continuum [23] 通过跨步骤前缀缓存共享提出内存优化, 以减少冗余预填充。但这些系统未能像 Pythia 一样, 充分利用可预测的程序语义进行细粒度服务优化。

9 结论

多智能体 LLM 工作负载的结构从根本上改变了服务系统的设计空间。通过暴露工作流语义, Pythia 将不透明请求转变为可预测执行, 解锁了传统系统无法实现的主动优化。我们的评估证明, 利用这种可预测性, 是克服缓存低效、资源不均衡和突发拥塞的关键。

图内文字的中文对照与译注

本节补全原图中嵌入的英文说明; 原图的数值、曲线、节点关系与系统名称保持原样。

工作流示意图。· 编码工作流的角色: Explorer = 探索者; Chronicler = 记录者; Architect = 架构师; Planner = 规划者; Engineer = 工程师; Reviewer = 审查者; Verifier = 验证者; Executor = 执行器。深度研究工作流: Decomposer = 分解器; Todo Manager = 待办管理器; Researcher #1/#2/#N = 研究员 1/2/N; Summarizer = 汇总器; General Search = 通用搜索; Academic Search = 学术搜索; Other Search = 其他搜索; Repeat a few times = 重复若干次。

缓存、时间线与资源问题示意图。· Token cache hits (k) = 缓存命中的词元数 (千); Request (%) = 请求比例; Time (seconds)/Time (s) = 时间 (秒); Event Sequence = 事件序列; Long Executor (36s) = 耗时较长的执行器 (36 秒)。Waiting requests = 等待中的请求数; GPU (tool use/code/reasoning) = 工具调用/代码/推理 GPU。KV Cache Space Utilization = KV 缓存空间利用率; Min-Max = 最小值至最大值; P25-P75 = 第 25 至第 75 百分位数; Median = 中位数; # of Preemptions = 抢占次数; # of Outstanding Requests = 未完成请求数。角色名称对应上一段。

系统概览图 (6)。· Multi-Agent Apps = 多智能体应用; Coding Assistant = 编码助手; Deep Research = 深度研究; Agent Frameworks = 智能体框架; Agent-Aware API Gateway = 感知智能体的 API 网关; API Interceptor = API 拦截器; Workflow Profiler = 工作流分析器; Global Request Scheduler = 全局请求调度器; Model Autoscaler = 模型自动扩缩容器; Cluster-Level = 集群层级; Worker-Level = 工作节点层级; Shared L3 Prefix Cache = 共享 L3

前缀缓存; Prefix Cache Manager = 前缀缓存管理器; Local Scheduler = 本地调度器; system annotation = 系统注释; logs = 日志; req = 请求; register = 注册。Pythia 为系统名称。

端到端性能图 (7)。· Coding Assistant = 编码助手; Deep Research = 深度研究; Mixed Workload (50/50) = 各占一半的混合工作负载; Avg. JCT (s) = 平均作业完成时间 (秒); P95 JCT (s) = 第 95 百分位作业完成时间 (秒); Throughput (tokens/s) = 吞吐量 (词元/秒); # Concurrent Workflows = 并发工作流数量。图例中的 Pythia、SGLang、vLLM、Autellix、Continuum、ThunderAgent 保留系统名称。

缓存效率图 (8)。· TTFT (ms) = 首词元时间 (毫秒); Decomposer(D) = 分解器 D; Summarizer(S) = 汇总器 S; Input Cache Hit Ratio = 输入缓存命中率; L1 (GPU) = L1 GPU 缓存; L2 (CPU) = L2 CPU 缓存; L3 (Storage) = L3 存储缓存; No hit = 未命中。+Sem. keep/drop = 增加语义感知的保留/丢弃; +Sem. drop +L2 pref. = 增加语义感知丢弃与 L2 预取; +Sem. drop +L1 pref. = 增加语义感知丢弃与 L1 预取。数值后的 s 表示秒, 未标注 s 的柱形数值与纵轴一致, 单位为毫秒。

排队动态与扩缩容图 (9、10)。· Tool use LLM = 工具调用 LLM; Code LLM = 代码 LLM; Reasoning LLM = 推理 LLM; Time (s) = 时间 (秒); 角色颜色对应工作流示意图。Queueing Delay (s) = 排队延迟 (秒); Static Equal = 静态等额分配; Phase Adaptive = 随阶段适配 (原图 Adaptive 拼写有误, 策略含义如此)。

突发性与预测准确性图 (11、12)。· Inter-arrival CV = 到达间隔的变异系数; Avg. JCT (s) = 平均作业完成时间 (秒); Throughput (tokens/s) = 吞吐量 (词元/秒)。LLM Requests (Sorted by Actual Completion Length) = LLM 请求 (按实际生成长度排序); Output Length (Tokens) = 输出长度 (词元); Predicted Allocation Interval = 预测分配区间; Actual Tokens (Ground Truth) = 实际词元数 (真实值)。

概率模型与原文表述的边界。· 本文由概率有限自动机生成的路径表达式含并行扇出、有界循环和可选后向边, 并不等同于严格无环的概率 DAG。统计容量推导在每个请求的越界概率确实受其给定上界约束时可应用联合上界; 历史经验第 99 百分位数本身并不自动保证未来、漂移后或对抗性输入仍满足该概率约束。正文保留原文的保证措辞, 讨论部分亦保留其冷启动与漂移回退建议。

源文件中的符号与实现细节。· 算法 3 在计算期望距离时写作 $E[D_a]$, 在累加风险分数时写作 $E[D_m]$; 算法 4 的输入写作 R_{curr} , 后续判断使用 $R[m]$ 。算法 2 使用 ExpectedSize(n) 计算余量, 正文余量式使用置信上界之和; 这些差异均按原文保留。正则表达式示例中的循环范围与可选分支也按原始字符串保留。算法中的函数名、角色键、JSON 键和提示占位符保留英文以维持程序语义; 提示中的自然语言和代码注释已译为中文。

参考文献

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *ACM SOSP*, 2023.
- [2] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient Execution of Structured Language Model Programs," in *NeurIPS*, 2024.
- [3] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve," in *USENIX OSDI*, 2024.
- [4] S. Yu, J. Xing, Y. Qiao, M. Ma, Y. Li, Y. Wang, S. Yang, Z. Xie, S. Cao, K. Bao, I. Stoica, H. Xu, and Y. Sheng, "Prism: Unleashing GPU Sharing for Cost-Efficient Multi-LLM Serving," 2025.
- [5] P. Steinberger, "OpenClaw." <https://openclaw.ai/>, 2026. Retrieved Mar 9, 2026.
- [6] M. Lee, S. So, and H. Oh, "Synthesizing regular expressions from examples for introductory automata assignments," *GPCE 2016*, 2016.
- [7] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations," in *Conference on Language Modeling*, 2024.
- [8] "LangGraph." <https://www.langchain.com/langgraph>, 2026. Retrieved Mar 9, 2026.
- [9] "LangChain." <https://www.langchain.com>, 2026. Retrieved Mar 9, 2026.
- [10] "OpenAI Python API library." <https://github.com/openai/openai-python>, 2026. Retrieved Mar 9, 2026.
- [11] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A Distributed Serving System for Transformer-Based Generative Models," in *USENIX OSDI*, 2022.
- [12] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," in *USENIX OSDI*, 2024.
- [13] "Claude Code bypassPermission Mode." <https://code.claude.com/docs/en/permission-modes>, 2026. Retrieved Mar 9, 2026.
- [14] "Codex Command Line Options." <https://developers.openai.com/codex/cli/reference>, 2026. Retrieved Mar 9, 2026.
- [15] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot," in *USENIX FAST*, 2025.
- [16] Y. Liu, Y. Cheng, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, R. Zhang, K. Du, and J. Jiang, "LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference," 2025.
- [17] "Coding Plan Overview." <https://www.alibabacloud.com/help/en/model-studio/coding-plan>, 2026. Retrieved Mar 9, 2026.
- [18] "ModelArk Coding Plan." <https://www.byteplus.com/en/activity/codingplan>, 2026. Retrieved Mar 9, 2026.
- [19] "MiniMax Token Plan." <https://platform.minimax.io/subscribe/token-plan>, 2026. Retrieved Mar 9, 2026.
- [20] "GLM Coding Plan." <https://z.ai/subscribe>, 2026. Retrieved Mar 9, 2026.
- [21] Y. Wu, S. Chen, Y. Zhong, R. Huang, Y. Tan, W. Zhang, L. Zhang, S. Zhou, Y. Liu, S. Zhou, M. Zhang, X. Jin, and P. Huang, "DualPath: Breaking the Storage Bandwidth Bottleneck in Agentic LLM Inference," 2026.
- [22] H. Kang, Z. Li, X. Yang, W. Xu, Y. Chen, J. Wang, B. Chen, T. Krishna, C. Xu, and S. Arora, "ThunderAgent: A Simple, Fast and Program-Aware Agentic Inference System," 2026.
- [23] H. Li, Q. Mang, R. He, Q. Zhang, H. Mao, X. Chen, H. Zhou, A. Cheung, J. Gonzalez, and I. Stoica, "Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live," 2026.
- [24] G. Ammons, R. Bodik, and J. R. Larus, "Mining specifications," in *ACM POPL*, 2002.
- [25] "vLLM Production Stack." <https://github.com/vllm-project/production-stack>, 2026. Retrieved Mar 9, 2026.
- [26] M. Luo, X. Shi, C. Cai, T. Zhang, J. Wong, Y. Wang, C. Wang, Y. Huang, Z. Chen, J. E. Gonzalez, and I. Stoica, "Autellix: An Efficient Serving Engine for LLM Agents as General Programs," 2025.
- [27] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. R. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *ICLR*, 2023.
- [28] A. Prabhakar, R. Ram, Z. Chen, S. Savarese, F. Wang, C. Xiong, H. Wang, and W. Yao, "Enterprise Deep Research: Steerable Multi-Agent Deep Research for Enterprise Analytics," 2025.
- [29] X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane, K. Sampath, M. Krishnan, S. Kundurthy, S. Hendryx, Z. Wang, V. Bharadwaj, J. Holm, R. Aluri, C. B. C. Zhang, N. Jacobson, B. Liu, and B. Kenstler, "SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?," 2025.
- [30] M. Du, B. Xu, C. Zhu, X. Wang, and Z. Mao, "DeepResearch Bench: A Comprehensive Benchmark for Deep Research Agents," 2025.
- [31] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, and X. Jin, "Fast Distributed Inference Serving for Large Language Models," 2024.
- [32] P. Patel, E. Choukse, C. Zhang, A. Shah, I. n. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient Generative LLM Inference Using Phase Splitting," in *ACM/IEEE ISCA*, 2025.
- [33] F. Strati, S. Mcallister, A. Phanishayee, J. Tarnawski, and A. Klimovic, "DéjàVu: KV-cache Streaming for Fast, Fault-tolerant Generative LLM Serving," in *ICML*, 2024.
- [34] R. Zhu, Z. Jiang, C. Jin, P. Wu, C. A. Stuardo, D. Wang, X. Zhang, H. Zhou, H. Wei, Y. Cheng, J. Xiao, X. Zhang, L. Liu, H. Lin, L.-W. Chang, J. Ye, X. Yu, X. Liu, X. Jin, and X. Liu, "MegaScale-Infer: Efficient Mixture-of-Experts Model Serving with Disaggregated Expert Parallelism," in *ACM SIGCOMM*, 2025.
- [35] K. Zhu, Y. Gao, Y. Zhao, L. Zhao, G. Zuo, Y. Gu, D. Xie, T. Tang, Q. Xu, Z. Ye, K. Kamahori, C.-Y. Lin, Z. Wang, S. Wang, A. Krishnamurthy, and B. Kasikci, "NanoFlow: towards optimal large language model serving throughput," in *USENIX OSDI*, 2025.
- [36] S. Agarwal, A. Mao, A. Akella, and S. Venkataraman, "Symphony: Improving memory management for llm inference workloads," 2024.
- [37] Z. Xie, Z. Xu, M. Zhao, Y. An, V. S. Maitlody, S. Mahlke, M. Garland, and C. Kozyrakis, "Strata: Hierarchical context caching for long context language model serving," 2025.
- [38] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, "FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving," in *MLSys*, 2025.
- [39] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *NeurIPS*, 2022.
- [40] Y. Xiang, X. Li, K. Qian, Y. Yang, D. Zhu, W. Yu, E. Zhai, X. Liu, X. Jin, and J. Zhou, "Aegaeon: Effective GPU Pooling for Concurrent LLM Serving on the Market," in *ACM SOSP*, 2025.
- [41] J. Duan, R. Lu, H. Duanmu, X. Li, X. Zhang, D. Lin, I. Stoica, and H. Zhang, "MuxServe: Flexible Spatial-Temporal Multiplexing for Multiple LLM Serving," in *ICML*, 2024.
- [42] Y. Fu, L. Xue, Y. Huang, A.-O. Brabete, D. Ustiugov, Y. Patel, and L. Mai, "ServerlessLLM: Low-Latency Serverless Inference for Large Language Models," in *USENIX OSDI*, 2024.
- [43] C. Lou, S. Qi, C. Jin, D. Nie, H. Yang, Y. Ding, X. Liu, and X. Jin, "HydraServe: Minimizing Cold Start Latency for Serverless LLM Serving in Public Clouds," 2025.
- [44] Z. Hu, Z. Pan, P. Kaur, V. Murthy, Z. Yu, Y. Guan, Z. Wang, S. Swanson, and Y. Ding, "Pancake: Hierarchical Memory System for Multi-Agent LLM Serving," 2026.

- [45] X. Tan, Y. Jiang, Y. Yang, and H. Xu, “Towards End-to-End Optimization of LLM-based Applications with Ayo,” in *ACM ASPLOS*, 2025.
- [46] Z. Pan, A. Patel, Z. Hu, Y. Shen, Y. Guan, W.-L. Li, L. Qin, Y. Wang, and Y. Ding, “KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows,” 2025.